

SUBHAKAMANA STORE

Progress Report - Production Readiness and System Improvement

Date: 2026-07-15

Prepared for: Subhakamana Store project demonstration and development handoff

Repository: tw849k8cz5-dot/subhakamana-store-

Executive Summary

The Subhakamana Store Management System is now organized as a complete front-end retail prototype with three main interfaces: Admin ERP, Physical Store POS, and Customer E-commerce Website. The system remains suitable for demonstration and project presentation, while production conversion has started through a documented PostgreSQL/Prisma database foundation and a new backend API foundation module.

The most important finding is clear: the user interface is strong enough to preserve, but real business use requires a backend, secure authentication, server-side permissions, transaction-safe inventory, reliable payment verification, backups, logs, and deployment controls. The current work therefore focuses on preserving the strongest workflows while progressively moving authority from the browser into production backend services.

Area	Current Progress	Status
Admin ERP	Dashboard, products, stock, orders, delivery, payments, returns, reports, role navigation, audit-style records, and workflow shortcuts are available in the prototype.	Prototype strong
Physical POS	Separate counter POS exists with search/scan, cart, manual price, discounts, payments, stock reduction, and cashier shift concepts.	Prototype strong
Customer Website	Storefront, product catalogue, cart, checkout, delivery/pickup, and demo payment/order records are present.	Prototype strong
Database	PostgreSQL and Prisma schema foundation exists for products, variants, inventory, orders, payments, returns, users, roles, sessions, and audit logs.	Module 1 started
Backend API	Dependency-free Module 2 API foundation added with health, RBAC rules, idempotency validation, and inventory safety rules.	Module 2 started
Production Readiness	Full P0/P1/P2/P3 audit and module reports have been added under production-readiness.	In progress

Work Completed So Far

- Built and refined the Subhakamana Store ERP prototype for stock, products, orders, delivery, payments, reports, returns, suppliers, customers, settings, and manual guidance.
- Built a separate Physical Store POS experience for counter billing and simplified cashier use.
- Built a customer-facing e-commerce website with product browsing, cart, checkout, delivery/pickup options, trust sections, and Subhakamana branding.
- Added local launcher files so the system can run without a domain for local demonstration.
- Added documentation, manuals, UML use case diagram, hosting report, framework report, QA reports, and full project summaries.
- Added ERP workflow improvements for fast physical sale, receive stock, check stock, duplicate product, payment verification, returns by bill, dispatch from order queue, and dashboard attention list.
- Added production-readiness architecture, API contract, security/RBAC plan, testing/deployment plan, migration plan, and database implementation plan.
- Added PostgreSQL/Prisma Module 1 database foundation and Module 2 backend API foundation starter.

Recent Repository Changes

File / Folder	Purpose
production-readiness/08-full-system-production-audit.md	Formal production audit with P0/P1/P2/P3 priority register.
production-readiness/09-module-02-api-foundation-implementation.md	Module 2 implementation status report.
production-readiness/module-02-api-foundation/	Backend starter with config, errors, health server, RBAC, idempotency, inventory rules, and tests.
production-readiness/module-01-database/scripts/verify-database-module.js	Fixed path handling so verification works when folder names contain spaces.
README.md and production-readiness/README.md	Updated links so coders can find the current audit and backend foundation.
subhakamana-store-final-erp-platform.html	ERP workflow improvements from the previous development step remain as uncommitted GitHub changes.

Audit Findings

Current Strengths

- The system already covers the three needed interfaces: Admin ERP, Physical POS, and Customer Website.
- The ERP is product-first, which matches the real store requirement to track products and stock rather than making the customer the center of inventory.
- The current UI is useful for staff training and presentation because common workflows are visible and understandable.
- Production planning is now documented with database schema, API contract, RBAC plan, migration plan, testing plan, and implementation reports.
- The Prisma schema already models key retail records such as users, roles, product variants, inventory ledger, orders, payments, deliveries, returns, refunds, stocktakes, cashier shifts, audit logs, notifications, settings, and idempotency keys.

Critical Launch Blockers

Priority	Issue	Meaning
P0	No live backend API connected to the app.	ERP, POS, and website cannot yet share real business data.
P0	No production PostgreSQL runtime connection.	The schema exists, but the running app still uses browser data.
P0	Critical data depends on localStorage.	Multiple staff or devices cannot safely use the system as one source of truth.
P0	Demo PIN and frontend-only roles.	Hiding buttons is not real security.
P0	Inventory is not transaction-safe.	POS and online checkout can conflict without database locking.
P0	Payments are simulated.	Payment success must be verified by backend/provider evidence.
P0	No tested backup/restore or production monitoring.	Business data recovery and issue visibility are not ready.

Production Modules

Module	Progress	Next Requirement
Module 1 - Database Foundation	PostgreSQL/Prisma schema exists and verification passes.	Run against a real PostgreSQL database and create migrations.
Module 2 - API Foundation	Health server, config, error shape, RBAC rules, idempotency validation, and inventory rules added.	Add real authentication, sessions, Prisma services, and route validation.
Module 3 - Product and Inventory API	Planned.	Move product, variant, SKU, barcode, stock receiving, and movement ledger to backend.
Module 4 - POS and Orders	Planned.	Make checkout atomic with idempotency and transaction-safe stock commit.
Module 5 - Website Checkout	Planned.	Reserve stock and create orders through backend instead of browser storage.
Module 6 - Payments	Planned.	Manual verification first; automated eSewa/Khalti later.
Module 7 - Returns and Shifts	Planned.	Link returns/refunds and cashier shifts to original backend records.

Tests and Validation

Test	Result	Note
Database module verification	Passed	Module 1 verifier passes after path fix.
Module 2 Node tests	Passed	5 tests passed for config, responses, RBAC, idempotency, and inventory oversell protection.
HTTP listen smoke test	Blocked	Sandbox prevented opening a local port with EPERM, so this was not counted as passed.
PDF generation and render check	Completed in this task	This report was generated and rendered for visual QA.

Immediate Next Steps

Step	Action	Reason
1	Commit the current GitHub changes after review.	Keeps the audit, backend foundation, and ERP workflow improvements safely recorded.
2	Run Module 1 against a real local PostgreSQL database.	Confirms schema migration works outside document-only planning.
3	Implement secure login and session storage.	Replaces demo PIN with real authentication.
4	Add server-side RBAC middleware.	Moves permissions from menu visibility to real backend enforcement.
5	Create product and inventory API endpoints.	Begins replacing localStorage with shared backend data.
6	Build transaction-safe POS checkout endpoint.	Prevents duplicate sales and stock conflicts.
7	Migrate one frontend workflow at a time.	Preserves the working prototype while moving toward production.

Management Conclusion

The project has moved from a broad local prototype into the beginning of a disciplined production-conversion path. The front-end system should be preserved because it already expresses the store workflow well. The next serious work is backend authority: authentication, permissions, PostgreSQL-backed products and inventory, atomic POS checkout, payment verification, and audit-safe records.

The system should not be described as production-ready yet. It should be described as a strong front-end prototype with a production database foundation, a new backend foundation starter, and a clear priority roadmap toward real business use.